

Les fonctions – partie 3

Tableaux et passage « par référence »

Une précision s'impose quant au passage des tableaux en paramètres d'une fonction. Dans le document précédent, nous avons mentionné que les tableaux sont toujours passés par référence et qu'ils ne sont donc jamais copiés. Il s'agit d'un abus de langage. Lorsque nous étudierons les pointeurs, nous verrons que lorsqu'un tableau est fourni comme argument, c'est en réalité l'adresse de son premier élément qui est transmise à la fonction. Cette adresse est elle-même passée par valeur, donc copiée. Grâce à cette adresse, la fonction peut néanmoins accéder aux éléments du tableau d'origine et les modifier.

```
void modifierTableau(int tab[], int taille)
// tab reçoit une copie de l'adresse du premier élément du tableau
{
    for (int i = 0; i < taille; i++)
    {
        tab[i] += 10;
    }
}
```

Nous verrons plus tard que, dans les paramètres d'une fonction, `int tab[]` et `int* tab` représentent le même type de paramètre.

Considérons cette définition de structure qui contient un tableau :

```
struct StructExemple
{
    int tab[5]{1, 2, 3, 4, 5};
};
```

Maintenant considérons cette définition de fonction :

```
void foo(StructExemple var)
{
    var.tab[0] = 100;
}
```

Considérons maintenant les instructions suivantes dans `main` :

```
StructExemple s;

foo(s);

cout << s.tab[0]; // affiche toujours 1
```

Après l'appel de cette fonction avec une structure de type `StructExemple` passée en argument, vous remarquerez que le membre `tab` n'aura pas changé. La valeur de `tab[0]` dans la structure sera toujours 1 au lieu d'être 100. Ceci s'explique parce que le paramètre `var` est passé par valeur. Ainsi, une copie complète de la structure passée en argument sera faite ce qui implique une copie distincte du tableau qu'elle contient. Pour que la fonction puisse modifier le tableau de la structure originale, nous pouvons passer la structure par référence, comme dans l'exemple qui suit.

Maintenant considérons cette nouvelle définition de fonction `foo` :

```
void foo(StructExemple& var)
{
    var.tab[0] = 100;
}
```

Considérons maintenant les instructions suivantes dans `main` :

```
StructExemple s;

foo(s);

cout << s.tab[0]; // affiche maintenant 100
```

La structure étant ici passée par référence, `var` désigne la structure d'origine. La modification de `var.tab` affecte donc directement le tableau qu'elle contient.

Surcharges de fonctions

La surcharge de fonctions consiste à définir plusieurs fonctions portant le même nom, mais ayant des paramètres différents. Lors d'un appel, le compilateur détermine quelle version utiliser selon les arguments fournis. Pour que des fonctions puissent être surchargées, elles doivent pouvoir être distinguées par le nombre de paramètres, leur type ou l'ordre de leurs types.

- Voici une surcharge selon le type d'un paramètre :

```
void foo(int valeur);
void foo(double valeur);

(...)

foo(5);        // appelle foo(int)
foo(5.2);      // appelle foo(double)
```

- Voici une surcharge selon le nombre de paramètres :

```
void foo(int valeur);
void foo(int valeur, int repetitions);

(...)

foo(5);        // appelle foo(int)
foo(5, 3);     // appelle foo(int, int)
```

- Voici une surcharge selon l'ordre des types paramètres :

```
void foo(int a, double b);  
void foo(double a, int b);
```

```
(...)
```

```
foo(5, 6.75);    //foo(int a, double b)  
foo(6.75, 3);    //void foo(double a, int b)
```

Le nom donné aux paramètres n'a aucune importance pour distinguer deux surcharges. Par exemple, les déclarations suivantes représentent la même signature et ne constituent donc pas une surcharge :

```
void foo(int a, double b);  
void foo(int x, double y); // erreur : même signature
```

Le type de retour ne permet pas de surcharger une fonction

Deux fonctions ne peuvent pas être distinguées uniquement par leur type de retour :

```
int foo(int valeur);  
double foo(int valeur); // erreur
```

Considérons cet appel :

```
foo(5);
```

Ici, le compilateur doit choisir la fonction avant même de considérer ce que nous ferons de la valeur retournée. Les paramètres des deux fonctions étant identiques, il ne peut pas les distinguer.

const et les paramètres passés par valeur

Ajouter `const` à un paramètre passé par valeur ne crée pas non plus une nouvelle surcharge :

```
void foo(int valeur);  
void foo(const int valeur); // erreur : même signature
```

Dans les deux cas, une copie de l'argument est transmise à la fonction. Le `const` indique seulement que la deuxième fonction ne pourrait pas modifier sa propre copie du paramètre.

Surcharge avec des références

Les références peuvent faire partie de la distinction entre certaines surcharges :

```
void foo(int& valeur);  
void foo(const int& valeur);
```

Ces deux fonctions peuvent coexister.

Considérons les appels suivants :

```
int a = 5;
const int b = 10;

foo(a);      // appelle foo(int&)
foo(b);      // appelle foo(const int&)
foo(20);     // appelle foo(const int&)
```

Une référence non constante (int&) peut seulement être associée à une variable modifiable.

Ainsi :

```
void modifier(int& valeur);

modifier(a);    // valide
modifier(20);   // erreur
```

Le littéral 20 ne représente pas une variable modifiable à laquelle une référence int& peut être associée.

Une référence constante (const int&) est différente :

```
void afficher(const int& valeur);

afficher(a);    // valide
afficher(b);    // valide
afficher(20);   // valide
```

Le mot-clé const signifie d'abord que la fonction s'engage à ne pas modifier la valeur par l'intermédiaire de cette référence. Cette propriété permet également à une référence constante d'être associée à une constante ou à une valeur temporaire, comme un littéral.

Surcharges valides, mais ambiguës à l'appel

Le fait que deux surcharges soient permises ne garantit pas que le compilateur pourra toujours choisir entre elles.

Considérons :

```
void foo(int valeur);
void foo(int& valeur);
```

Ces deux déclarations sont permises. Toutefois :

```
int a = 5;

foo(a);    // erreur : appel ambigu
```

Les deux fonctions peuvent recevoir a, et aucune ne constitue un meilleur choix suffisamment clair pour le compilateur.

Le même problème peut survenir avec :

```
void foo(int valeur);  
void foo(const int& valeur);  
  
foo(5);    // erreur : appel ambigu
```

Le littéral 5 peut être passé par valeur ou associé à une référence constante.

Ainsi, une surcharge peut être valide lors de sa déclaration tout en créant une ambiguïté lors de certains appels.

Paramètres par défaut

Il est possible de donner une valeur par défaut à un ou plusieurs paramètres d'une fonction. Cette valeur sera utilisée lorsque l'argument correspondant n'est pas présent à l'appel. Autrement dit, la valeur par défaut n'est utilisée que si l'argument est absent.

```
void afficher(int valeur, int repetitions = 1);
```

Les appels suivants sont donc tous les deux valides :

```
afficher(5);        // équivaut à afficher(5, 1) où 1 est la valeur par défaut  
afficher(5, 3);     // repetitions reçoit 3
```

Les paramètres par défaut doivent obligatoirement être placés à droite

Lorsqu'un paramètre possède une valeur par défaut, tous les paramètres qui le suivent doivent également en posséder une.

```
void foo(int a, int b = 10, int c = 20);    // valide  
void bar(int a = 10, int b);                // invalide
```

À partir de la déclaration de foo, nous pouvons alors effectuer les appels suivants :

```
foo(1, 2, 3);    // a = 1, b = 2, c = 3  
foo(1, 2);       // a = 1, b = 2, c = 20  
foo(1);          // a = 1, b = 10, c = 20
```

Il n'est cependant pas possible d'omettre un argument situé au milieu :

```
foo(10, ,30);    // appel invalide.
```

Paramètres par défaut, surcharge et appels ambigus

Les paramètres par défaut ne font pas partie de la signature d'une fonction. Ils peuvent cependant influencer les appels possibles et créer des ambiguïtés entre plusieurs surcharges.

Considérons les fonctions suivantes :

```
void foo(int valeur);  
void foo(int valeur, int repetitions = 1);
```

Les deux surcharges sont valides.

Cet appel ne pose aucun problème :

```
foo(5, 3); // appelle foo(int, int)
```

Par contre :

```
foo(5); // erreur : appel ambigu
```

Les deux fonctions peuvent répondre à l'appel. Le compilateur ne peut pas déterminer laquelle choisir.

Un autre exemple :

```
void foo(int valeur = 0);  
void foo(double valeur = 0.0);
```

Les surcharges sont valides et les appels suivants ne sont pas ambigus :

```
foo(5); // appelle foo(int)  
foo(5.5); // appelle foo(double)
```

Par contre :

```
foo(); // erreur : appel ambigu
```

Les deux fonctions peuvent être appelées sans argument grâce à leurs paramètres par défaut.

Attention à la combinaison des surcharges et des valeurs par défaut

Avant d'ajouter un paramètre par défaut à une fonction surchargée, il faut vérifier si cette valeur par défaut permet à plusieurs fonctions de répondre au même appel.

Par exemple :

```
void afficher();  
void afficher(int valeur = 0);  
  
afficher(); // ambigu
```

Une surcharge peut donc être valide lors de sa déclaration, mais certains appels peuvent devenir ambigus à cause des paramètres par défaut.

Valeurs par défaut dans la déclaration et la définition

Lorsqu'une fonction possède une déclaration et une définition séparées, la valeur par défaut est généralement indiquée dans la déclaration :

```
void afficher(int valeur, int repetitions = 1);  
  
void afficher(int valeur, int repetitions)  
{  
    // ...  
}
```

Il ne faut pas répéter inutilement la valeur par défaut dans la définition.

Tableau synthèse

	Variable int	Variable const int	Littéral / temporaire	Peut modifier l'original ?
foo(int valeur)	oui	oui	oui	Non
foo(int& valeur)	oui	non	non	Oui
foo(const int& valeur)	oui	oui	oui	non